Project: Database Creation in MS-SQL

Case Study: Relational Database Design for a Boutique Hotel Management System

---

Project Name: Hotel Rooms and Guests Management Database

1. Executive Summary & Project Overview

In the hospitality industry, operational efficiency and data accuracy directly impact the guest experience and revenue management. Manual tracking or poorly normalized legacy systems frequently lead to critical operational failures, such as double-bookings, uncoordinated room maintenance, and inconsistent guest tracking.

This case study details the development of a robust, highly normalized relational database management system (RDBMS) implemented in Microsoft SQL Server. The system is engineered to manage core hotel operations, including guest profiling, room inventory optimization, amenity allocation, and transactional reservation tracking.

2. Business Requirements & Problem Domain

To build an effective system, the database architecture had to address three core operational challenges:

- Guest Segmentation: The hotel categorizes guests into distinct types (e.g., Business Traveler, VIP, Remote Worker, Family) to offer tailored experiences and analytical insights into demographics.
- Dynamic Asset Configuration: Rooms vary by type (Suite, Deluxe, Standard) and are equipped with shifting assortments of facilities/amenities (Wi-Fi, Jacuzzi, Gym). The system must track these configurations cleanly without data redundancy.
- Reservation Lifecycle: The transactional core must accurately record check-in and check-out intervals, assign specific rooms, and dynamically store calculated baseline expenses (PerDayExpense) unique to each specific booking record.

3. Database Architecture & Schema Design

The system architecture utilizes a highly normalized schema satisfying Third Normal Form (3NF). This eliminates data modification anomalies and ensures strict referential integrity.

Data Dictionary & Structural Breakdown

- **The Guest Subsystem (GuestTypes, Guests):**
  Guests utilizes an Identity(501,1) auto-incrementing surrogate primary key for efficient indexing. It enforces data quality via a UNIQUE constraint on Email. A post-design structural optimization added a localized ***City char(2)*** attribute to support geographical marketing metrics. It maintains a many-to-one (N : 1) relationship with GuestTypes.

- **The Room Inventory (RoomsStatuses, RoomTypes, Rooms):**
  Rooms are decoupled from their types and real-time operational states. Rooms features foreign keys pointing to lookups, using a robust business default constraint (DEFAULT 101) to automatically initialize new rooms as "Available" or a baseline state.
- **The Many-to-Many Bridge (Facilities, Room_Facilities):**
  Because a single room can contain multiple facilities, and a facility can belong to multiple rooms, a direct relationship would violate normalization rules. A composite primary key PRIMARY KEY (RoomID, FacilityID) is implemented in the junction table Room_Facilities to map this relationship seamlessly. Additionally, Facilities utilizes a NONCLUSTERED primary key index to optimize specific query execution paths.
- **The Transactional Core (Bookings):**
  The Bookings table serves as the focal engine of the schema, binding a specific guest to a specific room across an explicit timeframe (CheckInDate to CheckOutDate).

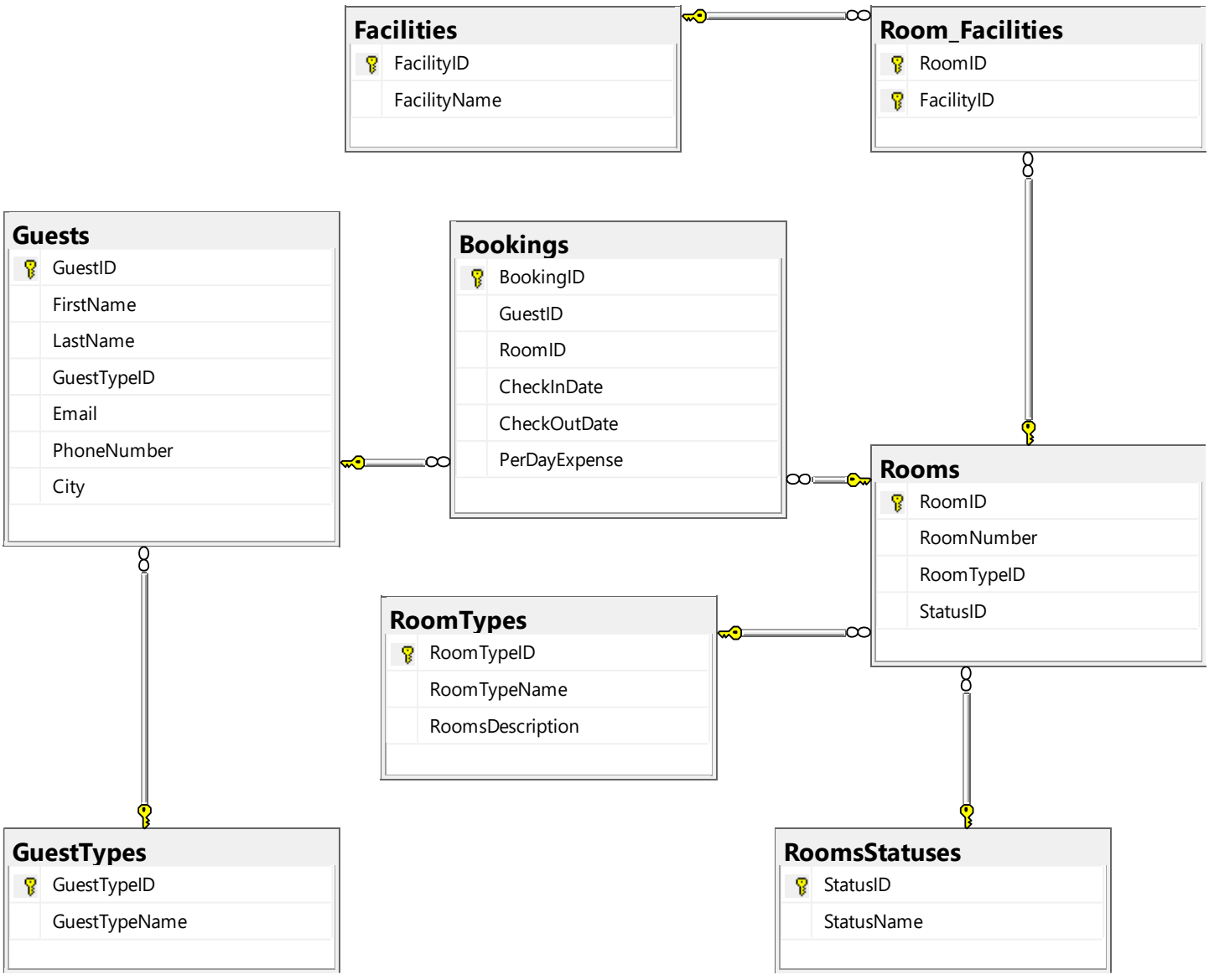


Fig: Database Diagram

4. Technical Implementation & Data Integrity

The SQL implementation enforces data safety at the database layer using explicit declarative constraints rather than relying on application-level code.

Schema Blueprint (DDL)

```sql
-- 1. Lookup Tables (Independent Entities)
CREATE TABLE RoomsStatuses (
    StatusID int PRIMARY KEY,
    StatusName varchar(50) NOT NULL
);

CREATE TABLE RoomTypes (
    RoomTypeID int PRIMARY KEY,
    RoomTypeName varchar(50) NOT NULL,
    RoomsDescription text
);

CREATE TABLE Facilities (
    FacilityID int PRIMARY KEY NONCLUSTERED,
    FacilityName varchar(50) NOT NULL
);

CREATE TABLE GuestTypes (
    GuestTypeID int PRIMARY KEY,
    GuestTypeName varchar(50) NOT NULL
);
```

```sql
-- 2. Master Tables (Dependent Entities)
CREATE TABLE Guests (
    GuestID int PRIMARY KEY IDENTITY(501, 1),
    FirstName varchar(50) NOT NULL,
    LastName varchar(50) NOT NULL,
    GuestTypeID int REFERENCES GuestTypes(GuestTypeID) NOT NULL,
    Email varchar(100) UNIQUE,
    PhoneNumber varchar(20)
);

-- Schema Evolution: Adding localized attributes
ALTER TABLE Guests ADD City char(2);

CREATE TABLE Rooms (
    RoomID int PRIMARY KEY,
    RoomNumber varchar(10) NOT NULL,
    RoomTypeID int REFERENCES RoomTypes(RoomTypeID) NOT NULL,
    StatusID int REFERENCES RoomsStatuses(StatusID) DEFAULT 101
);
```

```sql
-- 3. Junction & Transactional Tables
CREATE TABLE Room_Facilities (
    RoomID int REFERENCES Rooms(RoomID),
    FacilityID int REFERENCES Facilities(FacilityID) NOT NULL,
    PRIMARY KEY (RoomID, FacilityID)
);
```

```sql
CREATE TABLE Bookings (
    BookingID int PRIMARY KEY IDENTITY,
    GuestID int REFERENCES Guests(GuestID),
    RoomID int REFERENCES Rooms(RoomID),
    CheckInDate date NOT NULL,
    CheckOutDate date NOT NULL,
    PerDayExpense decimal(10, 2)
);
```

5. Operational Analytics & Verification

To validate the schema's effectiveness, complex analytical business queries can be effortlessly executed. The normalization structure allows management to generate high-value operational insights safely.

6. Conclusion & Future Roadmap

The engineered database provides a reliable, scalable foundation for the hotel’s digital infrastructure. By decoupling core entities and systematically mapping many-to-many relationships, the schema guarantees high data consistency.